

Sesión 04
Líneas de Productos Software

Anass El Jabiry Kaddir

Departamento de Informática
Universidad de Almería

Almería, lunes 11 de noviembre de 2024

Índice de Contenidos

Página

1. Introducción		3
2. Desarrollo Paso a Paso		3
Paso 1: Organización del Proyecto y Creación de Nuevas Vistas.....		3
Paso 2: Modificando VistaMain con TabView		5
Paso 3: Implementación de VistaIdentificacion.....		6
Paso 4: Implementación de VistaChat		7
Paso 5: Barra de Búsqueda en VistaListaAmigos		8
Paso 6: Gestión Dinámica de la Lista de Amigos		9
Paso 7: VistaAddAmigo.....		12
3. Conclusiones y Reflexiones		12

1. Introducción

En esta cuarta práctica, se amplió la funcionalidad de la aplicación **GestionAmigos** desarrollada previamente, añadiendo nuevas características como la gestión dinámica de la lista de amigos, el uso de un componente **TabView** para organizar vistas en pestañas, y la implementación de un chat simulado. Además, se añadió una vista de identificación de usuario, así como funcionalidades para buscar, añadir, mover y eliminar amigos de la lista.

El objetivo fue mejorar la experiencia de usuario, aportando flexibilidad y una organización más intuitiva de las distintas secciones de la aplicación.

Herramientas Utilizadas

- **Xcode 13.4.1**: Entorno de desarrollo integrado que permite crear y probar la aplicación en diferentes dispositivos iOS.
- **Swift**: Lenguaje de programación empleado para implementar lógica y funcionalidades avanzadas.
- **SwiftUI**: Framework utilizado para construir interfaces de usuario reactivas y adaptativas de manera eficiente.

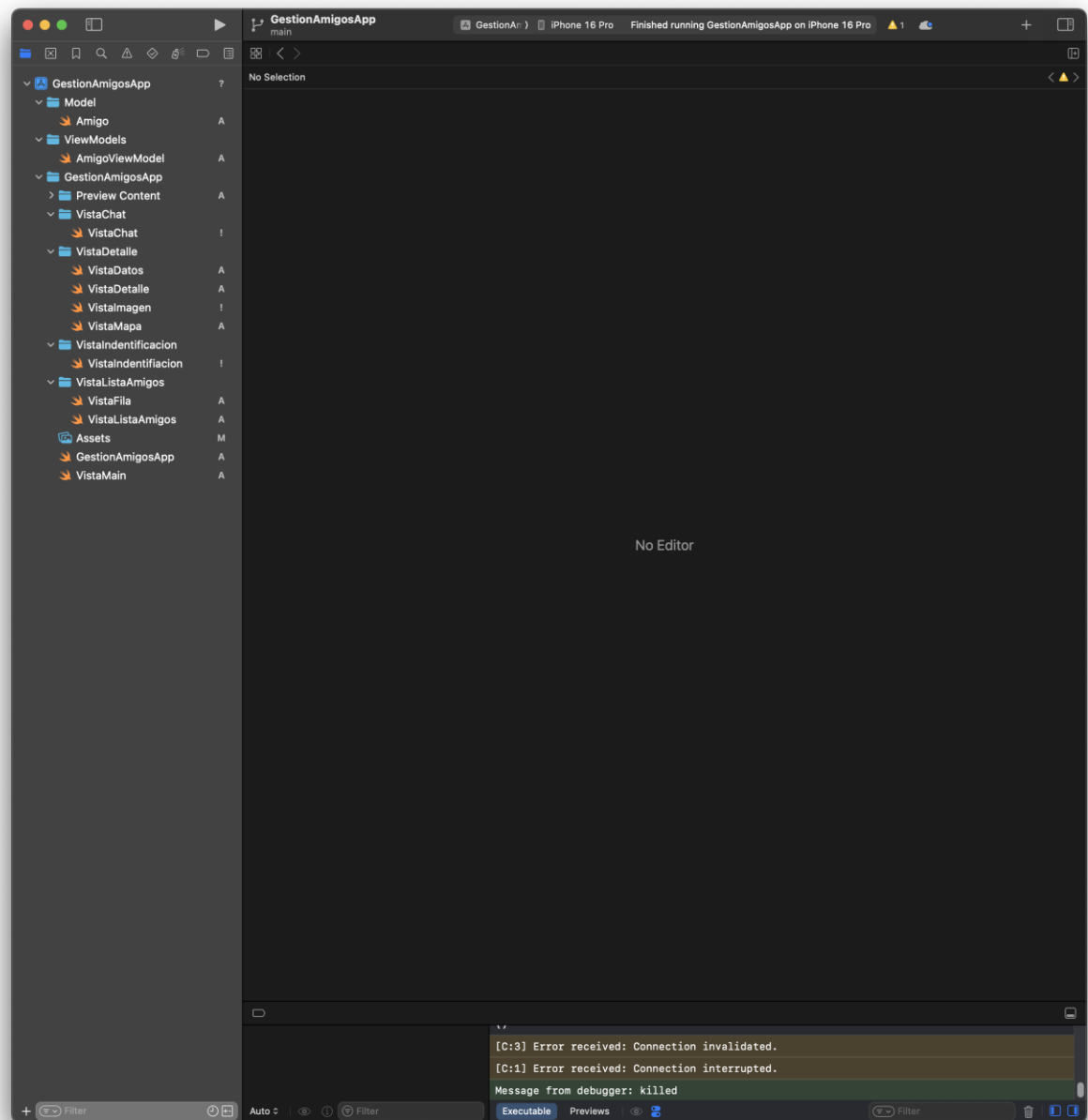
2. Desarrollo Paso a Paso

Paso 1: Organización del Proyecto y Creación de Nuevas Vistas

Se empezó por reorganizar el proyecto, siguiendo una estructura que separa el modelo de datos (Model), la vista (View), y el ViewModel, siguiendo el patrón **MVVM (Model-View-ViewModel)**. Esto facilita la mantenibilidad del código y mejora la separación de responsabilidades.

• Vistas Creadas:

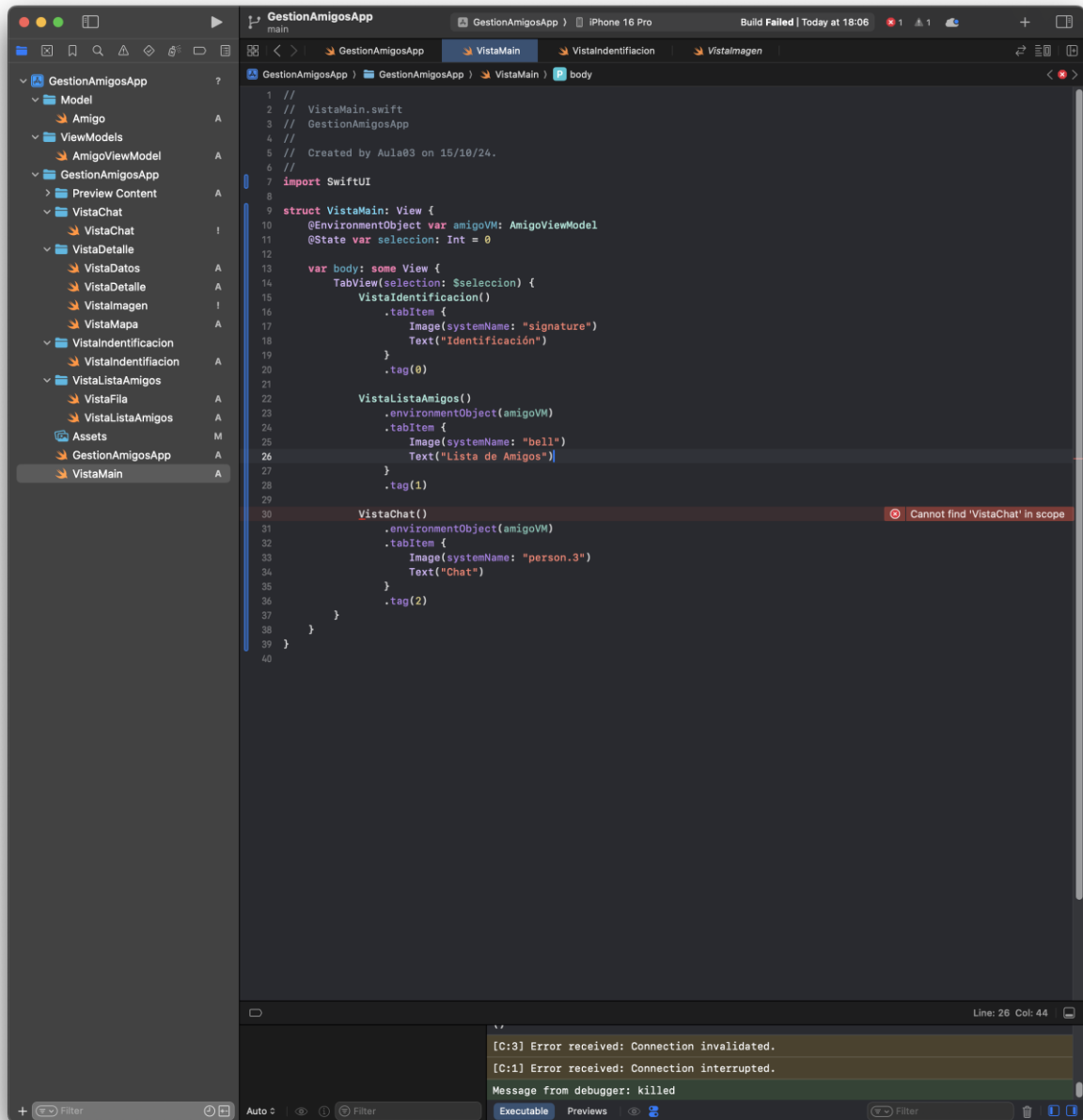
- **VistaMain**: La vista principal que organiza las distintas secciones mediante **TabView**.
- **VistaIdentificacion**: Se encarga de gestionar el inicio y registro del usuario.
- **VistaChat**: Simulación de un chat entre el usuario y sus amigos.



Captura 1: Proyecto reorganizado con grupos lógicos para cada componente de la arquitectura MVVM.

Paso 2: Modificando VistaMain con TabView

La vista principal de la aplicación, **VistaMain**, se implementó con **TabView** para mostrar las distintas secciones de la app. Cada pestaña se asoció con un tag numérico, lo cual permitió controlar programáticamente cuál de las vistas se debía mostrar, utilizando una variable de estado `@State` llamada `seleccion`.

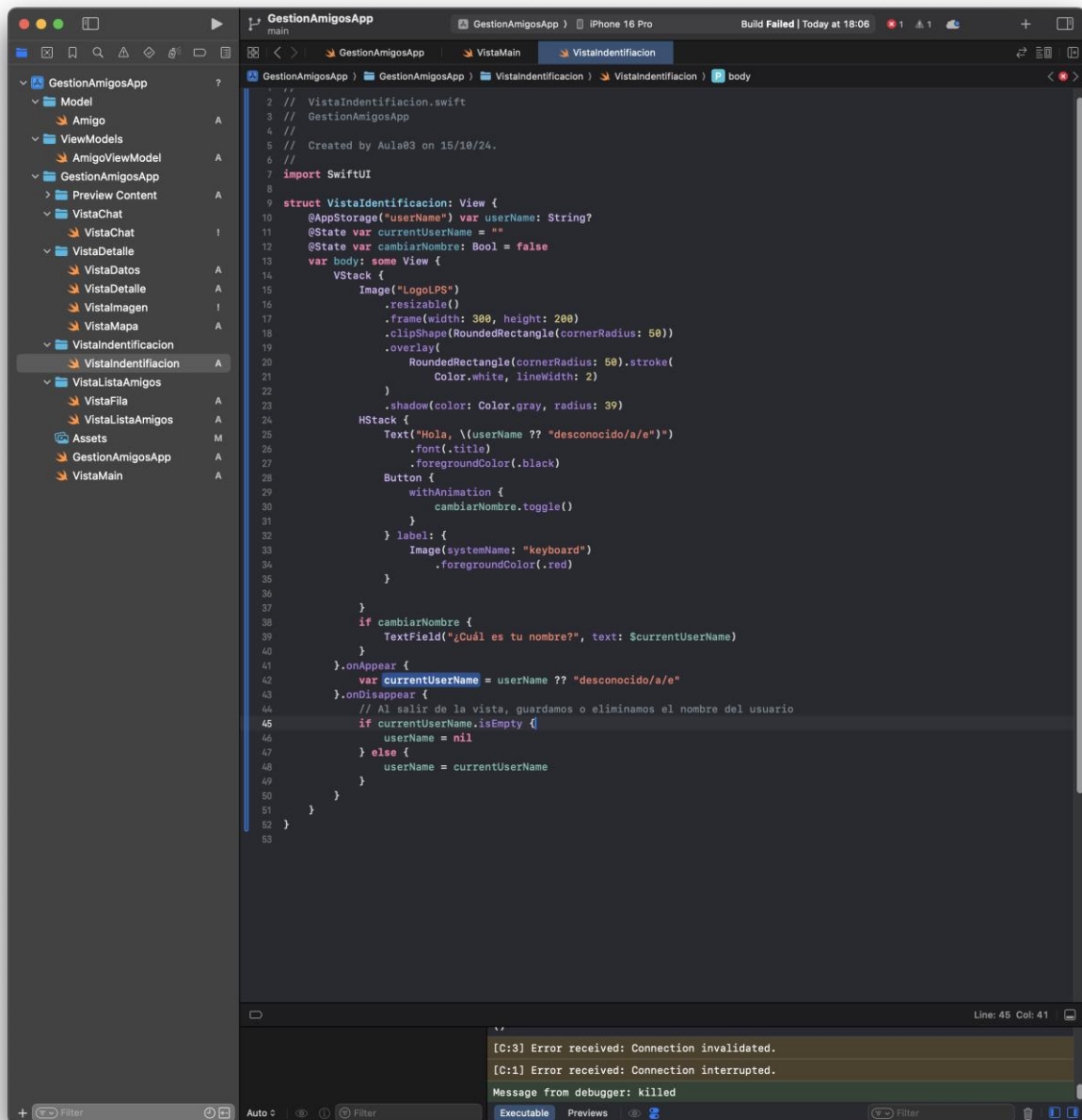


Captura 2: VistaMain mostrando la organización en pestañas con TabView.

Paso 3: Implementación de VistaIdentificacion

Se implementó una vista llamada **VistaIdentificacion**, que funciona como pantalla de bienvenida, donde el usuario puede registrarse y cambiar su nombre. Utilizamos `@AppStorage` para guardar el nombre del usuario en el dispositivo. Esto permite mantener la persistencia de dicho dato entre sesiones.

- **Problema y Solución:** Inicialmente se detectó un problema con el uso de `TextField` y una variable opcional `@AppStorage`. La solución fue usar `@State` para crear una variable temporal (`currentUserName`) que se inicializara en el evento `.onAppear()` y luego sincronizara los cambios con `userName` en el evento `.onDisappear()`.

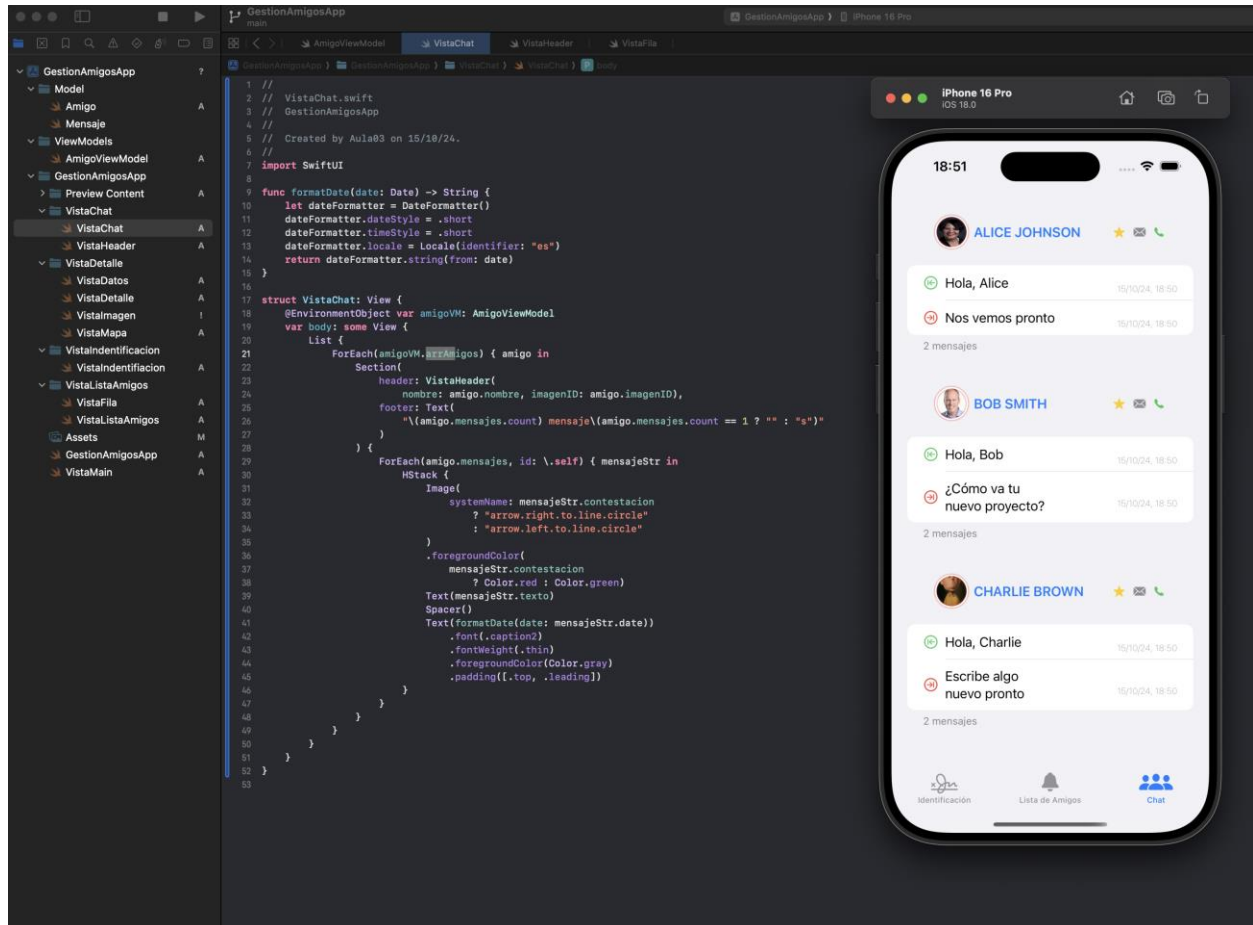


Captura 3: VistaIdentificacion mostrando el saludo al usuario y permitiendo la modificación del nombre.

- **Ejercicio 1:** Para que el nombre del usuario se quedara almacenado, se implementó un gestor de eventos `.onDisappear()`, el cual actualiza `userName` con el valor de `currentUserName`, pero se asegura de que si `currentUserName` está vacío, entonces `userName` sea `nil`.

Paso 4: Implementación de VistaChat

La **VistaChat** simula un chat entre el usuario y sus amigos. Se creó la estructura Mensaje, que se añadió al modelo de amigo para almacenar una lista de mensajes. El chat muestra los mensajes agrupados por amigo, permitiendo ver conversaciones ficticias.

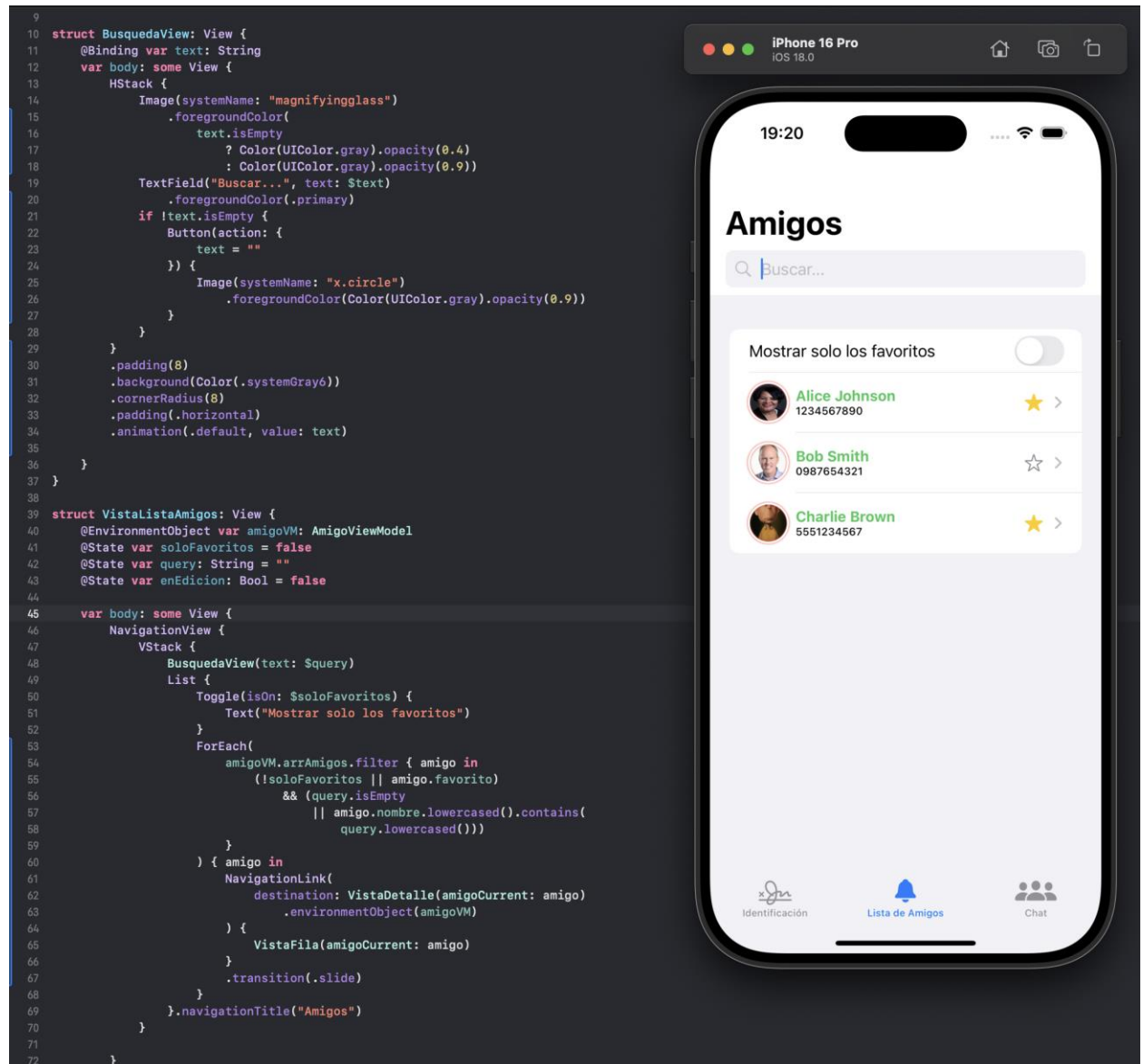


Captura 4: VistaChat mostrando la lista de mensajes agrupados por amigo, con iconos para diferenciar mensajes enviados y recibidos.

Paso 5: Barra de Búsqueda en VistaListaAmigos

Se implementó una barra de búsqueda personalizada dentro de **VistaListaAmigos** para facilitar la localización de amigos. La barra se implementó desde cero utilizando una subvista llamada **BusquedaView**, la cual contiene un **HStack** con un campo de texto y un icono de lupa.

- **Filtrado de Amigos:** La barra de búsqueda modifica el contenido de la lista filtrando los amigos cuyo nombre coincide con el texto ingresado en query. Además, se combinó con el filtro de favoritos (**soloFavoritos**) para permitir una visualización combinada.



Captura 5: Barra de búsqueda filtrando los amigos según el texto ingresado.

- **Ejercicio 2:** Se amplió la condición del if en el ForEach para tener en cuenta el filtro por nombre. El if finalizado se ve así:

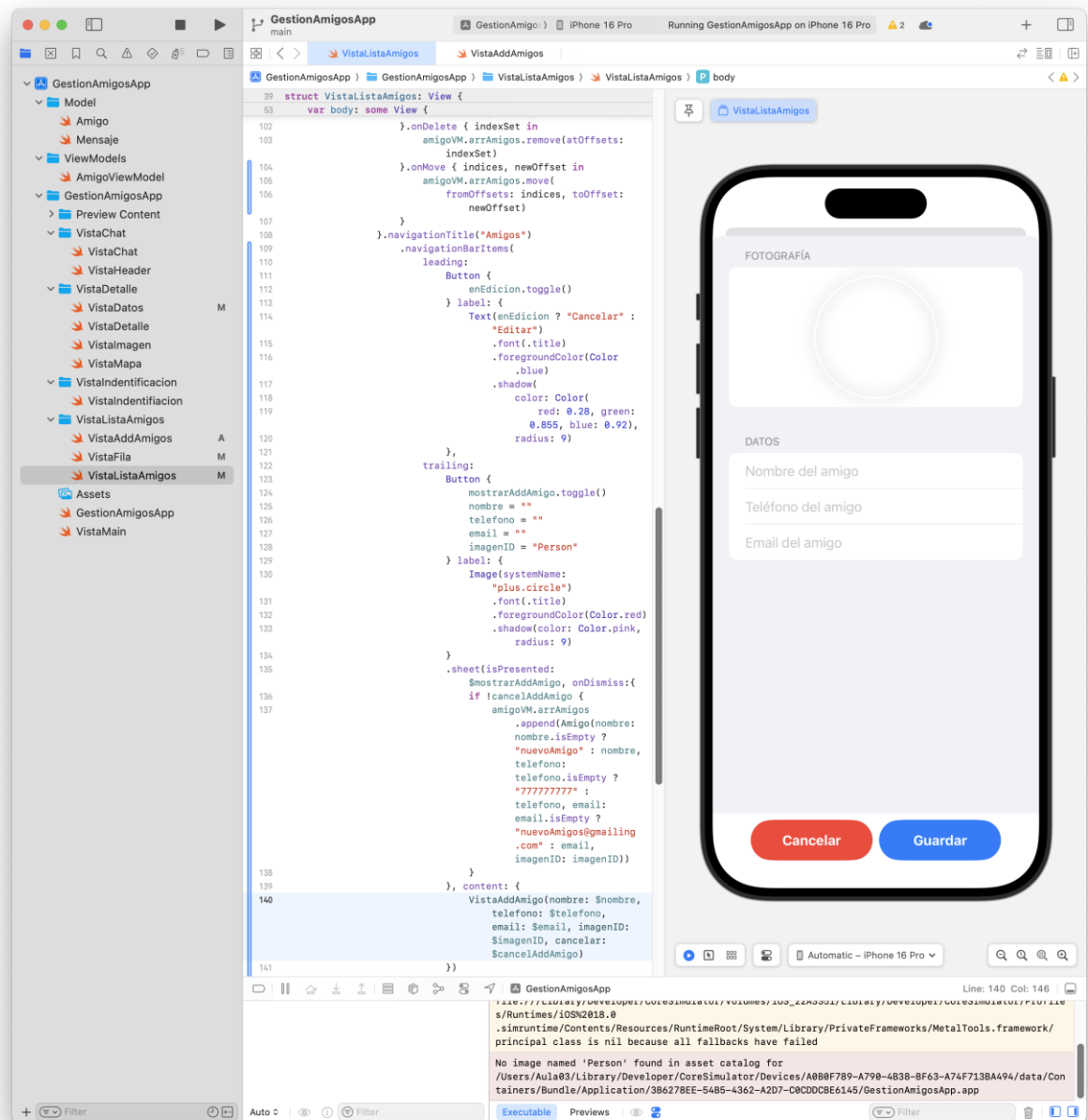
```
ForEach(
    amigoVM.arrAmigos.filter { amigo in
        (!soloFavoritos || amigo.favorito)
        && (query.isEmpty
            || amigo.nombre.lowercased().contains(
                query.lowercased()))
    }
) { amigo in
    NavigationLink(
        destination: VistaDetalle(amigoCurrent: amigo)
        .environmentObject(amigoVM)
    ) {
        VistaFila(amigoCurrent: amigo)
    }
    .transition(.slide)
}
}.navigationTitle("Amigos")
```

Se realizaron pruebas ingresando diferentes combinaciones de nombres y activando el filtro de favoritos para verificar que ambos filtros se aplicaban correctamente.

Paso 6: Gestión Dinámica de la Lista de Amigos

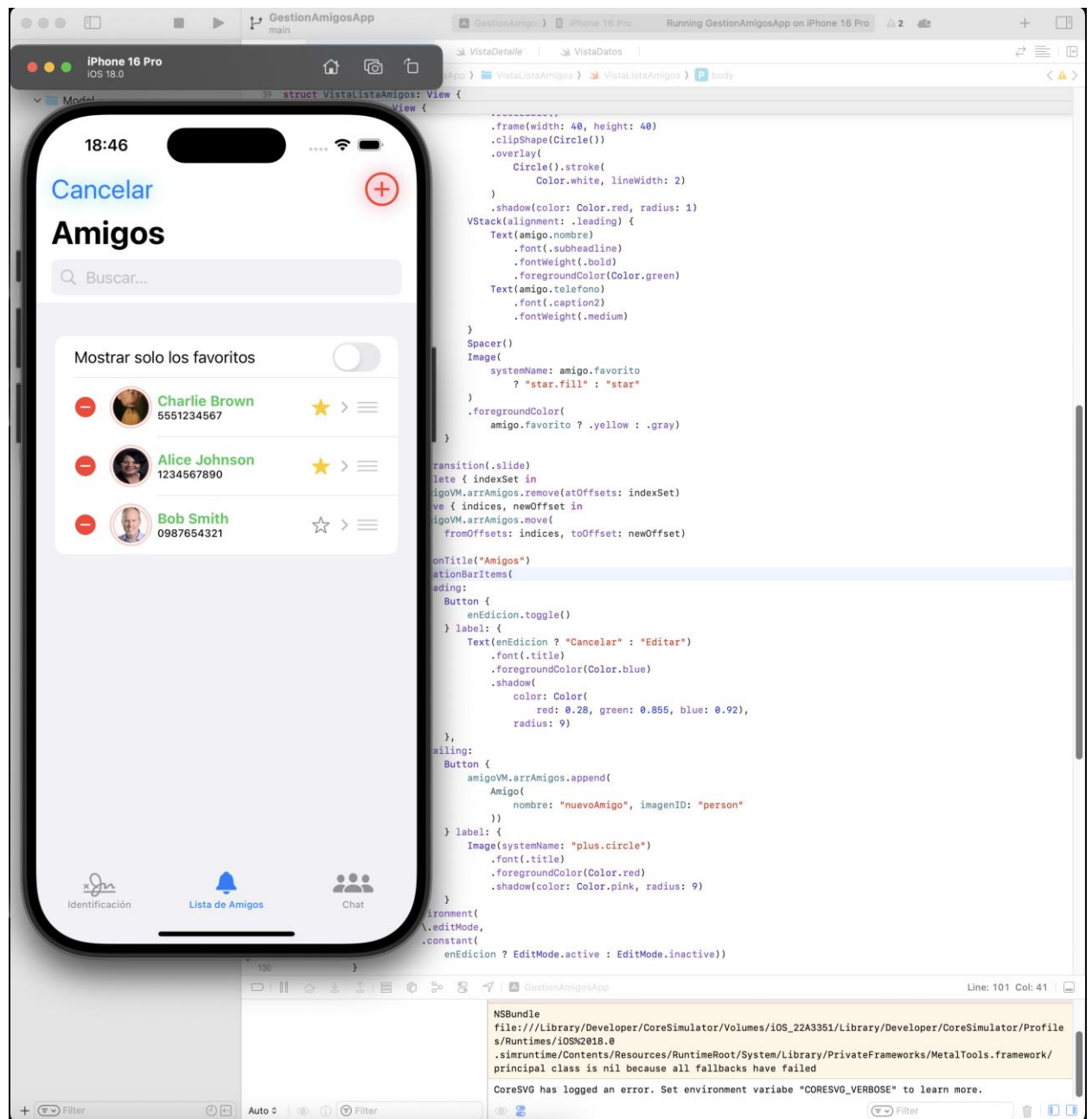
Se añadieron funcionalidades para eliminar, añadir y mover amigos dentro de la lista:

- **Eliminar Amigos:** Se utilizó el modificador `.onDelete()` sobre el ForEach principal de `VistaListaAmigos` para permitir la eliminación de amigos deslizando hacia la izquierda. Además, se realizaron cambios en el código para asegurarse de que la aplicación no se bloquea al eliminar un registro tras editarlo.
- **Mover Amigos:** Para permitir que el usuario cambie el orden de los amigos, se implementó el evento `.onMove()`, el cual reordena el array de amigos.
- **Editar Amigos:** Se habilitó la edición de los atributos nombre, telefono, email e imagenID al crear una vista intermedia (`VistaAddAmigo`) que permite agregar un nuevo amigo especificando esos atributos.



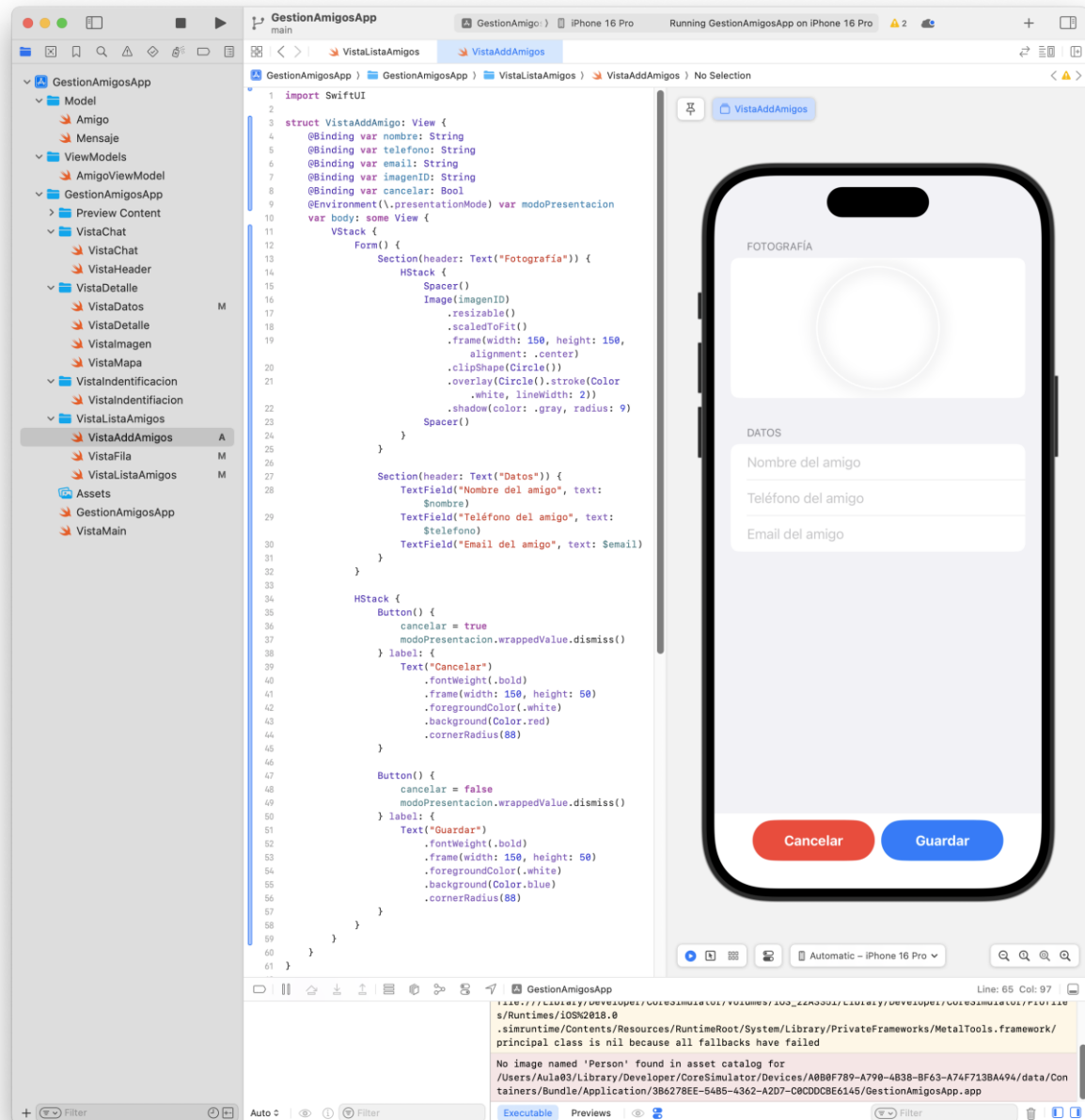
Captura 6: VistaListaAmigos mostrando la lista en modo edición, con opciones para mover y eliminar amigos.

- **Ejercicio 3:** Implementación del evento `.onMove()` y pruebas realizadas para verificar que los amigos se pueden mover libremente en la lista.



Paso 7: VistaAddAmigo

Se implementó la vista **VistaAddAmigo** para permitir la inserción de nuevos amigos. La vista consta de un formulario (Form) con campos para el nombre, teléfono, email e imagen. Los botones **Cancelar** y **Guardar** gestionan la acción correspondiente, cerrando la vista mediante `.dismiss()`.



Captura 7: VistaAddAmigo mostrando el formulario para agregar nuevos amigos.

- **Ejercicio 4:** Reflexionamos sobre la calidad del código y se propusieron algunas mejoras, como refactorizar código duplicado y utilizar extensiones para componentes que se repiten. Se sugirió también la implementación de validaciones adicionales para los campos del formulario, asegurando que no queden vacíos.

3. Conclusiones y Reflexiones

Esta práctica nos permitió llevar la aplicación **GestionAmigos** a un nuevo nivel, haciendo la lista de amigos completamente dinámica, con la capacidad de agregar, eliminar y mover elementos. Aprendimos a utilizar `TabView` para organizar vistas en pestañas, mejoramos la gestión de

persistencia con `@AppStorage`, e implementamos una simulación de chat para enriquecer la experiencia de usuario.

La organización del código siguiendo el patrón **MVVM** y la separación de las vistas en archivos y grupos lógicos hicieron que el proyecto fuera más mantenible y escalable. Finalmente, se implementaron varias mejoras para asegurar que los cambios realizados en los amigos se reflejan adecuadamente y que la app se comporta de forma robusta y predecible.